
Alpha-GPT 3.0: A Multi-Agent Framework for Quantitative Investments

Ronald Liu

University of California, Berkeley
ronald-liu@berkeley.edu

Abstract

An *alpha* is a formula that scores every stock each day, so that high-scoring stocks are expected to outperform low-scoring ones. Searching for good alphas is hard: the space of possible formulas is far too large to enumerate, and testing one candidate requires an expensive simulation over years of historical data. Automated methods such as genetic programming search this space mechanically, with no sense of which formulas are economically plausible.

We ask whether a large language model can supply that missing judgment. Alpha-GPT 3.0 runs two rounds of structured debate between LLM agents. In the first round the agents write no formulas at all, and argue in plain English about why a proposed effect should predict returns. In the second they turn the agreed idea into a formula. They then check each other's work: does the formula say what the idea said, and can it be computed from the data we have? Genetic programming then varies the survivors locally. Cheap automatic checks reject broken candidates before any simulation runs.

One \$99 run turned 456 natural-language ideas into 2,352 verified formulas. Combined equally into a single long-short portfolio, they earn a Sharpe ratio (return per unit of risk) of 1.10 on 2021–2025 data that was held out from the entire search. A portfolio built from accounting data could earn this simply by tracking familiar effects such as value or profitability, so we also strip out its exposure to the full set of input characteristics; what remains still earns 1.18, at less than half the drawdown. The same run exposes the method's main limitation. The generated formulas resemble one another, with an average pairwise correlation of 0.15, and a portfolio of correlated bets behaves like only about $1/0.15 \approx 7$ independent ones. The portfolio therefore stops improving after the first hundred or so formulas: further progress requires formulas that differ from one another, not more of them.

1 Introduction

Quantitative equity investors rank stocks using formulas called *alphas*. An alpha reads market and accounting data and gives every stock a score each day. The investor buys the highest-scoring stocks and sells the lowest-scoring ones. The strategy makes money if the ranking is even slightly better than chance. A simple example, written in the notation this paper uses, is

$$\text{sub}(\text{cs_rank}(\text{cs_roa}), \text{cs_rank}(\text{cs_accruals})),$$

which favors companies that are profitable but whose reported earnings are backed by cash rather than by accounting accruals. Any single alpha is weak at best, so practitioners combine hundreds of them: many weak and largely unrelated signals average into one that is considerably more reliable.

Finding such formulas is hard for three reasons. First, the space of candidate formulas is astronomically large, because operators can be nested to arbitrary depth. Second, scoring one candidate means

simulating it over years of historical data, which is slow. Third, and most damaging, a formula that fits historical data well is usually just lucky, so a search that simply maximizes historical performance will mostly return noise.

The established response is mechanical search. Genetic programming and reinforcement learning generate formulas, mutate the best ones, and keep whatever scores highest on historical data [9, 33, 34]. These methods cover ground efficiently, but they have no notion of *why* a formula should work. They cannot tell a formula that captures a real economic mechanism from one that fits the same data by coincidence. So they overfit easily, and the alphas they produce are hard for a human to interpret.

Large language models can supply what mechanical search lacks. A model trained on the finance literature already knows that firms with low accruals have tended to outperform, and that small illiquid stocks behave differently from large ones. It can also state that reasoning in words. Financial LLMs such as BloombergGPT and FinGPT confirm that models carry substantial domain knowledge [20, 21], and LLM judgments about news carry some return-predictive information [11]. Alpha-GPT and Alpha-GPT 2.0 applied this to alpha mining, using an LLM to help a human researcher turn ideas into formulas [16, 24]. The obvious next step is to ask a model to write alpha formulas on its own. In practice this is unreliable. The formula it returns often drifts away from the economic idea it was meant to express. It may refer to data fields that do not exist in the dataset, or fail to parse at all. Such candidates are discarded before they are ever tested, so the model call is wasted.

Our fix is to stop asking for both things at once. Alpha-GPT 3.0 splits the work into two separate conversations, each held by three LLM agents given different investment perspectives (a momentum agent, a mean-reversion agent, and a fundamental agent). In the first conversation, the *idea debate*, the agents may not write any formula. They must argue in plain English about the economic mechanism: what effect is being claimed, in which direction it should move returns, and which data fields could measure it. They score and criticize each other’s arguments, revise their own in response, and a moderator merges what survives into a few written hypotheses. In the second conversation, the *formula debate*, the agents turn each hypothesis into candidate formulas. They then review one another on two questions. Does this formula express what the hypothesis claimed? Can it be computed from the data fields this run provides? Settling the economics first has a practical payoff. Every formula can be traced back to three things: the argument that motivated it, the agent that wrote it, and the reviews it survived. When an alpha behaves strangely, we can read the reasoning that produced it instead of guessing.

We then measure what this generator produces. Two things are easily mistaken for real skill: performance that only reflects well-known return effects, and a large alpha count that hides heavy duplication. We test for both. Our contributions are:

- **A two-stage debate generator** (Section 4). Agents settle the economics before any formula is written, then check each other’s formulas both for faithfulness to the idea and for whether they can actually be computed. Every alpha it emits carries a record of the idea and the reviews behind it.
- **A precise statement of the problem** (Section 3). We write alpha mining as an optimization problem over a grammar of formulas, and use Grinold’s fundamental law to explain why many weak, unrelated alphas beat a few strong ones. This is what justifies aiming for breadth in the first place.
- **An evaluation that controls for known effects** (Section 5). Beside every headline number we report the same number computed after removing the portfolio’s exposure to the full set of input characteristics, rather than to a handful of standard factors, which is a much stricter control.
- **A measured ceiling on breadth** (Section 5). From one \$99 run we show that the effective number of independent bets, $N_{\text{eff}} = N/(1 + (N - 1)\bar{\rho})$, can never exceed $1/\bar{\rho}$, and that our portfolio already reaches 99.6% of that ceiling with 1,485 alphas. Generating more alphas therefore cannot help; generating less similar ones is the only lever left.

Section 2 places the work among related lines of research. Section 3 states the problem precisely and Section 4 describes the system. Section 5 reports what one run produced, and Section 6 sets out the changes those results call for.

2 Related Work

Four lines of work bear on this paper: LLMs in finance, alpha search before LLMs, LLMs writing alphas, and multi-agent argument. A fifth line, LLM trading agents, is easy to confuse with ours, so we mark the difference explicitly.

Large language models in finance. Financial language models have moved from task-specific encoders such as FinBERT [1] to general models adapted to the domain. BloombergGPT shows that pretraining on a mixture of financial and general text improves performance on finance tasks [20], and FinGPT provides an open-source route to the same end [21]. Separately, studies that feed news to ChatGPT-style models find the resulting judgments carry some information about future returns [11]. This work establishes that the models know finance, but it targets prediction, sentiment, and question answering rather than the writing of formulas.

Searching for alphas before LLMs. Until recently the search was purely mechanical. AlphaGen uses reinforcement learning to assemble sets of formulas that work well together [33]. AutoAlpha uses a hierarchical evolutionary algorithm [34]. AlphaEvolve learns to discover new alphas [35]. AlphaForge mines formulas and also adjusts how they are combined over time [36]. The hand-built counterpart is Kakushadze’s “101 Formulaic Alphas” [8], and the underlying technique goes back to Koza’s genetic programming [9]. Related work in asset pricing shows that flexible machine-learning models predict returns better than linear ones [6]. These methods are the baseline we are trying to improve on: they fit historical data well, but nothing in them prefers an economically sensible formula over a coincidental one.

Using LLMs to write alphas. Our direct predecessors are Alpha-GPT and Alpha-GPT 2.0. They put a human researcher and an LLM in a loop to turn ideas into formulas [16, 24]. The present system is the autonomous version of that idea. The closest concurrent work is CogAlpha [25]. There, a hierarchy of agents evolves Python *code* rather than symbolic formulas, on the argument that formulas are too restrictive. It uses price and volume data only, and aims for a small number of high-quality alphas. Others in this group take different routes. Kou et al. build a multi-agent strategy finder [26]. AlphaAgent adds pressure against alphas that stop working once widely known [27]. AlphaJungle searches formulas using Monte Carlo tree search [28]. RD-Agent(Q) optimizes the factors and the model that consumes them together [29].

LLMs inside evolutionary search. The machinery these systems borrow comes from outside finance. FunSearch uses an LLM as the mutation step of an evolutionary program search [30]. AlphaEvolve builds that into a coding agent for discovering algorithms [31]. Mind Evolution treats evolutionary search as a way of spending more computation at inference time [32]. We use the same division of labor in a cheaper form: the LLM makes the large, informed moves and ordinary genetic programming [5] makes the small local ones at no model cost.

Making models argue. A separate line asks how to get more out of a fixed model by structuring its reasoning. Chain-of-thought prompting and self-consistency show that reasoning step by step, and sampling several times, improves reliability [18, 17]. Tree-of-Thoughts and ReAct search over intermediate states [22, 23]. Self-Refine and Reflexion show that letting a model criticize and revise its own output helps without any retraining [12, 14]. Systems such as CAMEL, AutoGen, ChatDev, MetaGPT and AgentVerse divide work across several interacting agents [10, 19, 13, 7, 3]. Debate between agents improves factual accuracy [4] and the reliability of model-based evaluation [2]. We apply this to alpha generation by separating debate about *what to test* from debate about *how to write it down*.

A different problem: LLM trading agents. Systems such as FinMem, TradingGPT and FinAgent use LLMs to make trading decisions, that is, what to buy and when. That is a different task from writing the formula that ranks the stocks in the first place, and we mention it only to mark the boundary.

How this paper differs. Taking CogAlpha as the closest comparison, we differ on five points. Our agents argue adversarially from three fixed viewpoints, instead of being organized into a coverage

hierarchy. We divide the work by stage, idea and then formula, rather than by market topic. We generate symbolic formulas rather than code: they are faster to evaluate, easy to parse, safe from look-ahead by construction, and open to genetic programming. We use ordinary genetic programming for local search instead of spending further model calls on it. Finally, we use accounting fundamentals as well as price and volume, on a survivorship-free U.S. panel. We aim for many weak alphas rather than a few strong ones, for the reason given in Section 3.6.

3 Problem Formulation

This section gives the complete formal apparatus: the data objects, the expression language, the evaluation functional, and the optimization problem the generator solves.

3.1 Data: panels, universe, splits

The market is represented as a library of *panels*. A panel for data field k is a matrix

$$X^{(k)} \in \mathbb{R}^{T \times N}, \quad X_{t,i}^{(k)} = \text{value of field } k \text{ for stock } i \text{ on day } t,$$

indexed by trading date t and security identifier (PERMNO) i . All fields share one date-by-stock grid, so they can be combined cell-wise; missing cells are NaN and every operator is NaN-safe.

The investable set on day t is the point-in-time universe $\mathcal{U}_t \subseteq \{1, \dots, N\}$ (roughly 1,500 names, re-selected annually by trailing market capitalization; survivorship-free, names retained through delisting). Price/volume fields come from CRSP daily data; fundamental characteristics come from Compustat quarterly filings and the WRDS financial-ratio suite, dated by their *release* date (earnings-announcement date `rdq`, or the WRDS `public_date`, which we verified is lagged a median of 92 days after quarter-end with zero violations) and forward-filled to daily, so mixing frequencies never introduces look-ahead.

The prediction target. For each stock and day the *forward return* is

$$y_{t,i} = r_{t+1,i} = \frac{P_{t+1,i}}{P_{t,i}} - 1,$$

the next-day total return with delisting returns embedded.

Splits. The calendar is partitioned *once* into two blocks:

$$\underbrace{2010\text{--}2020}_{\text{in-sample (selection)}} \quad | \quad \underbrace{2021\text{--}2025}_{\text{test (held out)}}.$$

Selection of everything (which expressions survive, which combiner is highlighted) happens in-sample; no selection decision ever uses test data. The generator’s headline result reads the test split once, through the final portfolio backtest; the additional diagnostics reported beside it (the factor-neutral residual and the gate sweep) are exploratory reads of that same window and are labelled as such. The last in-sample forward return is blanked (embargoed) so no label leaks across the boundary.

Remark 3.1 (Why two splits, not three). An earlier configuration used train/validation/test with a sign-consistency check across train and validation. A single long in-sample block strictly dominates for our selection rule: the gate below is a t -statistic on the mean daily IC, whose power grows as $\sqrt{T}$, and eleven years of daily cross-sections give the highest-power test the data allows. A longer window is also harder to overfit by construction: a signal must have worked across multiple regimes to pass.

3.2 Alpha expressions

Definition 3.1 (Alpha expression). An *alpha expression* e is a well-typed symbolic tree whose leaves are *terminals* (data panels, drawn from a finite menu $\mathcal{T}$) and whose internal nodes are *operators* (drawn from a finite set $\mathcal{O}$). Evaluating the tree produces a *signal panel*

$$\alpha = e(X) \in \mathbb{R}^{T \times N}, \quad \alpha_t = (\alpha_{t,i})_{i \in \mathcal{U}_t} \in \mathbb{R}^{N_t}.$$

The operator set is small and fixed (the “alpha language”):

- **Time-series** (per stock, along time; curried with fixed windows $w \in \{5, 10, 20, 60\}$): `ts_mean`, `ts_std`, `ts_delta`, `ts_rank`, `ts_min`, `ts_max`, `ts_returns`, and the binary `ts_corr`.
- **Cross-sectional** (per day, across stocks): `cs_rank`, `cs_zscore`.
- **Element-wise**: `add`, `sub`, `mul`, `safe_div`, `log_abs`, `abs_val`, `sign`, `neg`.

A concrete example, “recent return scaled by volatility, ranked cross-sectionally”:

$$e = \text{cs_rank}(\text{safe_div}(\text{ts_returns_20}(\text{close}), \text{ts_std_20}(\text{returns}))).$$

Why the space is intractable. With typical operator arity b and maximum tree depth D , the number of admissible expressions grows roughly as

$$|\mathcal{E}| \sim (|\mathcal{T}| + |\mathcal{O}|)^{\Theta(b^D)}, \quad (1)$$

doubly exponential in depth. The search is over *programs* (terminals, operators, and nesting), so direct enumeration is infeasible and a guided search is required.

Remark 3.2 (Formula vs. code representation). The tree fixes one *level* of representation: compositions of the bounded operator set $\mathcal{O}$. A deeper alternative, central to CogAlpha [25], lets the generator emit general *code* (intermediate variables, regime logic, transforms outside $\mathcal{O}$), reaching the strictly larger $\mathcal{E}_{\text{code}} \supset \mathcal{E}_{\text{formula}}$. We default to the DSL because it is fast, parseable, and open to free genetic amplification, whereas code needs sandboxing, timeouts, and a look-ahead audit, evaluates more slowly, and forfeits free GP; and because only this backend changes, “generate code” drops in as a third arm for an all-else-equal comparison the literature lacks.

3.3 From a signal to cross-sectional bets

A signal is used *cross-sectionally*: each day we rank stocks by α_t and bet that high-score names outperform low-score names. On day t each valid stock is assigned to one of $Q = 5$ quantiles by the rank of its score,

$$q_{t,i} = \left\lceil Q \cdot \frac{\text{rank}(\alpha_{t,i})}{N_t} \right\rceil \in \{1, \dots, Q\},$$

with tie-breaking so bins never collapse. The long–short quintile portfolio is long the top bucket, short the bottom bucket, equal-weighted:

$$R_t^p = \underbrace{\frac{1}{|L_t|} \sum_{i \in L_t} y_{t,i}}_{\text{long top quintile}} - \underbrace{\frac{1}{|S_t|} \sum_{i \in S_t} y_{t,i}}_{\text{short bottom quintile}}, \quad \begin{aligned} L_t &= \{i : q_{t,i} = Q\}, \\ S_t &= \{i : q_{t,i} = 1\}. \end{aligned} \quad (2)$$

Because positions are determined by *rank*, only the cross-sectional ordering of α_t matters; any monotone transform of a signal is the same bet. Days with no cross-sectional dispersion take no position, and per-leg daily returns are clipped to $[-95\%, +100\%]$ to bound the impact of data errors.

3.4 Measuring one signal: IC, ICIR, and the t -statistic gate

Definition 3.2 (IC, ICIR, t -statistic). For expression e with signal $\alpha = e(X)$, the daily information coefficient is the Spearman rank correlation between scores and forward returns,

$$\text{IC}_t(e) = \rho_{\text{Spearman}}(\alpha_t, y_t) = 1 - \frac{6 \sum_i d_i^2}{N_t(N_t^2 - 1)}, \quad \overline{\text{IC}}(e) = \frac{1}{T} \sum_{t=1}^T \text{IC}_t(e), \quad (3)$$

where d_i is stock i ’s rank difference. The stability-adjusted versions are

$$\text{ICIR}(e) = \frac{\overline{\text{IC}}}{\text{std}_t(\text{IC}_t)}, \quad t(e) = \text{ICIR}(e) \sqrt{T}, \quad (4)$$

the t -statistic of the mean daily IC over the T in-sample days.

$\overline{\text{IC}}$ measures average directional accuracy (persistently above ≈ 0.02 is economically meaningful); ICIR is its signal-to-noise; $t(e)$ turns that into a calibrated significance statement. When IC is computed on a k -day subsample for speed, the t -statistic is rescaled by $\sqrt{k}$ so the stored value always reflects the full in-sample length. We also track *turnover*,

$$\tau(e) = \frac{1}{T} \sum_t \frac{1}{N_t} \sum_i |u_{t,i} - u_{t-1,i}|, \quad u_{t,i} = \frac{\text{rank}(\alpha_{t,i})}{N_t}, \quad (5)$$

a proxy for rebalancing cost, plus coverage and structural statistics.

The selection gate. An expression enters the portfolio iff

$$|t(e)| \geq t_{\min} \quad (\text{default } t_{\min} = 2), \quad (6)$$

i.e. its mean in-sample IC is statistically distinguishable from zero. This replaces the earlier train/validation sign-consistency filter with a single higher-power test. Daily ICs are serially correlated, so (4) treats them as independent only approximately; we use $t(e)$ as a ranking statistic and coarse gate, not a calibrated p -value. Section 5 measures what this gate buys out of sample.

3.5 Combining signals and their direction

A single alpha is weak; the edge comes from combining many weak, weakly-correlated signals (the *breadth* of Section 3.6). Given the surviving set $S = \{e_1, \dots, e_M\}$, each signal is cross-sectionally standardized,

$$s_t^{(j)} = z(\alpha_t^{(j)}), \quad z(\alpha_t)_i = \frac{\alpha_{t,i} - \text{mean}(\alpha_t)}{\text{std}(\alpha_t)}, \quad (7)$$

and the composite is the equal-weight blend $g_t = \frac{1}{M} \sum_j s_t^{(j)}$.

Two deliberate design decisions here differ from common practice and from an earlier version of this system:

Remark 3.3 (No sign-flipping: native orientation only). Many pipelines flip each signal by the sign of its in-sample IC so that “all point the profitable way.” We do not. Flipping uses in-sample returns to choose the trade direction (data-mining the sign), and for a hypothesis-driven alpha it is incoherent: the economic reasoning *is* a directional claim, and a formula whose native direction loses in-sample is a *falsified hypothesis*, not a free sign correction. Signals therefore trade in the orientation the expression itself encodes, and the *flip rate* (the fraction of proposed alphas whose in-sample IC is negative) becomes a falsification metric for the generator: a rate near 50% means the economic reasoning has no directional skill; materially below 50% means the direction was genuinely predicted.

Remark 3.4 (Combiner: equal weight, not fitted tangency). The Sharpe-maximizing weights for fixed S are the tangency portfolio $w^* \propto \Sigma^{-1}\mu$; with many correlated, noisy alphas the sample Σ^{-1} is ill-conditioned and w^* overfits. Equal weight is the maximal-shrinkage limit, beats sample tangency out of sample, and keeps any measured edge attributable to the alphas rather than to combiner tuning. (IC-magnitude weighting was implemented and removed: it adds an in-sample fitted parameter for no reliable out-of-sample gain.)

3.6 The objective and the optimization problem

Running the composite through (2) yields daily returns R_t^p ; the headline objective is the annualized test-split Sharpe ratio

$$\text{SR} = \sqrt{252} \frac{\text{mean}_t R_t^p}{\text{std}_t R_t^p} \quad (8)$$

(risk-free rate zero, standard for a dollar-neutral book whose short-leg financing nets against the long leg). We also report annualized return, maximum drawdown, and the composite’s own IC/ICIR/ τ .

The generator chooses the set $S \subseteq \mathcal{E}$; everything downstream is fixed by scope, so S is the sole decision variable:

$$\max_{S \subseteq \mathcal{E}} \mathbb{E}[\text{SR}_{\text{test}}(\text{portfolio}(S))] \quad \text{s.t.} \quad \begin{cases} \text{every } e \in S \text{ parses and passes the verifier,} \\ S \text{ is chosen on in-sample data only (6).} \end{cases} \quad (9)$$

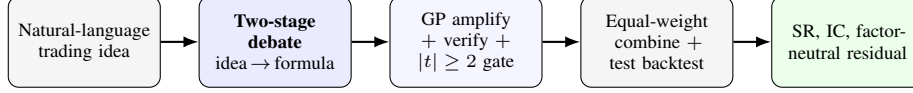


Figure 1: The pipeline. Each of the two debate stages is a draft → cross-review → revise loop over three persona agents, closed by a moderator (Section 4.5). Everything to the right of the generator (GP amplification, verification, the t -statistic gate, combination, and backtest) is held fixed across all generation arms (random, single-shot LLM, debate), so a difference in the output metrics is attributable to the proposal mechanism alone.

The expectation matters because the optimizer never sees SR_{test} : it maximizes an in-sample surrogate, and test is read exactly once. That in-sample-to-test gap is the central statistical risk. The problem is hard for two reasons: $\mathcal{E}$ is doubly exponential (1), and each evaluation is an expensive backtest. We can neither enumerate nor differentiate, only *search*.

What the objective rewards: skill and breadth. For a dollar-neutral long–short book the portfolio return *is* the active return, so its Sharpe equals the information ratio, and Grinold’s fundamental law factors it into the two levers a proposer controls:

$$SR = IR \approx \underbrace{IC}_{\text{skill}} \cdot \sqrt{\underbrace{\text{breadth}}_{\# \text{ independent bets}}} \quad (10)$$

Skill enters linearly but breadth only under a square root, so many diverse, modest-IC alphas beat a few strong ones; the combiner accordingly keeps S large with no top- k cap. Equation (10) is not only a design principle; it is the key to *interpreting* results, because breadth alone can produce a high Sharpe from common premia that require no skill. For this reason Section 5 reports performance orthogonal to the full characteristic basis alongside every headline number.

4 Method: generating alphas by debate

Since (9) can be neither enumerated nor differentiated, we search. The search is a generate-and-test loop (Figure 1) built to spend cheap effort widely and the expensive true objective narrowly.

4.1 Propose globally, refine locally, prune cheaply

1. **Propose (global moves).** The multi-agent debate takes an economically reasoned hypothesis and writes *complete* expressions for it. Because it draws on the model’s pretrained economic knowledge, it lands directly in plausible regions of $\mathcal{E}$ instead of wandering in from random starts.
2. **Refine (local moves).** Each proposal can seed genetic programming, which mutates and recombines it (subtree crossover, point mutation) to hill-climb in-sample IC in a small neighborhood: many nearby variants at no extra model cost.
3. **Prune (cheap first, expensive last).** The debate’s cross-review drops economically weak ideas; the structural verifier discards anything that fails to parse, evaluate, or vary (including a static degeneracy check for expressions that collapse algebraically, e.g. $\text{sub}(x, x)$); fast approximate metrics rank the rest; only survivors of the t -gate (6) reach the full backtest.

The division of labor is the point: the LLM makes large, economically informed jumps a local search never could, and GP does cheap refinement the LLM would waste calls doing by hand. The proposer is *fixed*: it does not learn from which past alphas backtested well; the only feedback it receives during a debate is *return-blind* diagnostics (coverage, rank turnover, distinct values, dead days) so critique is grounded in implementability without becoming IC-tuning.

4.2 Agent design

Three specialized agents participate in both stages, each initialized with a distinct investment prior: the **Momentum** agent reasons from trend persistence, price strength, breakout dynamics, and volume

confirmation; the **Mean-Reversion** agent from overshooting, contrarian positioning, and volatility-induced corrections; the **Fundamental** agent from valuation, quality, and the slower incorporation of accounting information into prices. Despite these stylistic differences each agent is a complete researcher, expected to reason across signal role, data-field selection, directionality, normalization, neutralization, and implementability under the operator set. Three structurally distinct priors surface a wider range of interpretations than single-agent sampling while keeping the number of LLM calls tractable. All inter-agent communication is structured JSON rather than free text, and every debate persists its full state as an artifact; Appendix A reproduces an abridged example.

4.3 Stage 1: the idea debate

Given a natural-language trading idea, Stage 1 produces two to three structured research-hypothesis specifications through a three-phase process:

Drafting. Each agent independently writes an idea proposal with an explicit economic mechanism, intended signal role (cross-sectional vs. time-series), required terminals, predicted direction, subfactors or conditioning filters, and a normalization/neutralization plan. Agents are constrained to the terminals actually available in the data panel and may not embed numerical formulas at this stage.

Cross-review. Each agent reviews the other two proposals across eight scored dimensions (mechanism quality, signal-type clarity, payoff clarity, directionality, subfactor quality, filter logic, normalization soundness, implementability) and renders accept / accept-with-revision / reject with free-form comments; the exact instruction is shown in Figure 2.

Review the following idea proposals written by other agents:
{proposals_json}

For each proposal, provide exactly one review object.
Return a JSON array where each item has:

```
{
  "target_proposal_id": "...",
  "mechanism_quality": 1-5,
  "signal_type_clarity": 1-5,
  "payoff_clarity": 1-5,
  "directionality_clarity": 1-5,
  "subfactor_quality": 1-5,
  "filter_logic": 1-5,
  "normalization_soundness": 1-5,
  "implementability": 1-5,
  "decision": "accept|accept_with_revision|reject",
  "comments": ["...", "..."]
}
```

Figure 2: The Stage 1 cross-review instruction (verbatim template, abridged; brace placeholders are filled per call). Fixed integer scales and a ternary decision make review outcomes comparable across debates and auditable afterward.

Revision and synthesis. Each agent revises its own proposal, explicitly recording which feedback was accepted and which rejected; this bookkeeping is what makes downstream failure analysis attributable to specific critiques. A moderator then synthesizes the three revised proposals into the final hypothesis specifications, instructed to *converge* (preserve meaningful distinctions, remove near-duplicates) rather than introduce new ideas, and to annotate each output hypothesis with its source agents.

4.4 Stage 2: the formula debate

Stage 2 turns hypothesis specifications into concrete symbolic formulas through an analogous three-phase debate. In drafting, each agent writes one to two candidates per hypothesis over the shared operator/terminal registry, tagged with a formula role and a rationale connecting expression to hypothesis. In cross-review, candidates are scored on faithfulness to the originating hypothesis, implementability, robustness, novelty relative to standard factors, and simplicity, with particular attention to whether the formula preserves the intended *direction* (which, by Remark 3.3, is binding) and avoids subtle look-ahead through aggregation windows. In revision-and-selection, agents revise, every formula passes a hard *parse gate* against the live operator/terminal registry, verifier-rejected formulas are dropped, and the moderator selects a final seed set (up to eight) with an explicit

preference for diversity across hypotheses, emitting a traceability map from each selected formula back to its hypothesis and source agent.

Algorithm 1 Two-stage debate: idea $\rightarrow$ formula (one proposal round)

Require: trading idea $\mathcal{H}$, terminal menu $\mathcal{T}$, agents $\{1, 2, 3\}$, moderator Φ

Stage 1: idea debate

- 1: each agent **drafts** a structured hypothesis (mechanism, direction, proxies, filters, normalization)
- 2: each agent **cross-reviews** the others on 8 dimensions; decision $\in \{\text{accept, revise, reject}\}$
- 3: each agent **revises** given the reviews
- 4: $\mathcal{H}^* \leftarrow \Phi(\text{revised hypotheses})$ $\triangleright$ moderator synthesizes ≈ 3 hypotheses

Stage 2: formula debate

- 5: each agent **drafts** 1–2 expressions per hypothesis over $\mathcal{T}$
 - 6: each agent **cross-reviews** on $\{\text{faithfulness, implementability, robustness, novelty, simplicity}\}$
 - 7: each agent **revises**; **parse-gate** + verifier drop non-compiling or degenerate expressions
 - 8: **return** seed pack $\leftarrow \Phi(\text{parseable expressions})$ $\triangleright$ moderator selects ≤ 8 diverse seeds
-

Cost is roughly 20 model calls per idea (three agents $\times$ three phases $\times$ two stages, plus two moderator calls); ideas are independent, so debates parallelize freely across a thread pool.

4.5 Why debate rather than the alternatives

Diversity, then critique. Search quality under a fixed proposal budget is set by the *best* candidates found. One model sampled many times returns the same few ideas reworded: the budget re-draws a single mode. Three persona-conditioned proposers pull toward different economic mechanisms, so the same budget covers more of $\mathcal{E}$; cross-review then adds what a plain sampler cannot, killing weak proposals before any of them costs a backtest. Debate buys *exploration* and *verification* at once. Against the alternatives: self-consistency collapses many chains onto one consensus answer (we want a diverse *set*); best-of- N has no persona diversity and no critique; MCTS needs a cheap value estimate for partial programs, and here the only ground truth is the expensive backtest.

The skill-breadth hypothesis. The fundamental law (10) says a good proposer needs out-of-sample skill *and* breadth. Table 1 lays out where we *expect* each mechanism to land (predictions, not measurements). The priorless methods can fit the training data, but with no economic reasoning to anchor them their best in-sample fits tend to be overfit; debate is built for both genuine skill (economic priors plus cross-review) and breadth (cheap, diverse proposals).

Mechanism	Skill	Breadth	Why we expect this
Random trees	none	high	unlimited cheap trees, but no economic prior; <i>individually</i> meaningless
Brute-force enum.	low	very low	no prior, so high in-sample IC is mostly overfit; reaches only a shallow sliver of $\mathcal{E}$
Genetic programming	low	low	hill-climbs in-sample IC with no economic grounding; explores only locally
Single-shot LLM	moderate	moderate	an economic prior gives a genuine but unrefined edge; one call stays on a single theme, no critique
Multi-agent debate	high	high	priors + cross-review favor genuinely predictive alphas; diverse personas widen coverage

Table 1: *Hypothesized* skill and breadth by proposal mechanism. *Skill* is genuine out-of-sample per-alpha IC; *breadth* is the number of independent alphas supplied. The grid is the planned comparison; this paper runs the debate arm (Section 5).

4.6 The outer loop and how we evaluate

To attribute any edge, the debate runs against baselines that strip the economic prior and the multi-agent structure (random expression trees, single-shot LLM generation, and unseeded GP), each through an *identical* downstream pipeline (verify, evaluate, gate, combine, backtest), with GP amplification as an on/off toggle. Reading down a column of the grid isolates the economic prior;

Algorithm 2 Outer search: propose $\rightarrow$ amplify $\rightarrow$ verify $\rightarrow$ evaluate $\rightarrow$ store

Require: rounds N , in-sample split, primitive set

```
1:  $\mathcal{P} \leftarrow \emptyset$   $\triangleright$  the book: every expression found, deduplicated on hash
2: for  $n = 1, \dots, N$  do
3:   seeds  $\leftarrow$  PROPOSE( $\mathcal{H}_n$ )  $\triangleright$  Algorithm 1, or single-agent / random
4:   candidates  $\leftarrow$  seeds  $\cup$  GP-AMPLIFY(seeds)
5:   for each  $e$  in candidates do
6:     if not VERIFY( $e$ ) then continue  $\triangleright$  parse, depth, eval, coverage, non-constant, non-degenerate
7:   end if
8:   compute  $\overline{\text{IC}}(e)$ , ICIR( $e$ ),  $t(e)$ ,  $\text{SR}^{\text{is}}(e)$ ,  $\tau(e)$  in-sample; insert into  $\mathcal{P}$ 
9: end for
10: end for
11:  $S \leftarrow \{e \in \mathcal{P} : |t(e)| \geq t_{\min}\}$   $\triangleright$  the gate (6); no top- $k$  cap
12: z-score each  $e \in S$  on test (7)  $\triangleright$  native orientation; no flipping
13: backtest the equal-weight composite on test (2)–(8); report alongside the factor-neutral diagnostic (Section 5)
```

across a row, local amplification. The sharpest single test is single-agent+GP vs. debate-only: at matched downstream effort it separates structured multi-agent reasoning from merely spending more compute on one proposer.

All arms run on the survivorship-free 1,500-name panel with the two-split design of Section 3.1, the $|t| \geq 2$ gate, native orientation, and the equal-weight combiner. Every run reports the portfolio comparison table (equal-weight composite, best-single-alpha reference, and the factor-neutral diagnostic (11)), the gate-ablation sweep, the generation funnel (generated $\rightarrow$ verified $\rightarrow$ gated) with per-reason reject counts, validity / duplicate / flip rates, and, for LLM arms, token-metered API cost per stored alpha. *Validity* (the fraction of candidates that parse and verify) measures the implementability dividend of parse-gating; *flip rate* (Remark 3.3) measures directional skill. An earlier pilot on a smaller three-split configuration already showed that unstructured single-shot generation degrades sharply when its seeds are invalid or misaligned, which motivated this structured, attributable design. Section 5 reports the debate arm measured under this protocol.

5 Results

We ran the pipeline once, end to end: 456 natural-language ideas were debated, translated into formulas, verified, evaluated in-sample, and combined under the protocol of Section 4.6, with everything downstream of the generator held fixed.¹ Table 2 summarizes the run’s inputs, outputs, and cost, and Figure 3 compresses it into the attrition chain from ideas to effective bets. All performance numbers below are out-of-sample on the held-out test split (2021–2025) and gross of transaction costs; the combination method (equal weight) and the selection gate ($|t| \geq 2$) were chosen in-sample.

Ideas submitted	456	Total cost	\$99.20
Debates completed	454	Cost per debate	\$0.22
Model calls	9,145	Cost per unique alpha	\$0.042
Prompt tokens	41.0M	Expressions emitted	3,253
Completion tokens	15.2M	Unique verified alphas	2,352
Generation wall-clock	41 min	Survivors ($ t \geq 2$)	1,485

Table 2: Run statistics. The model is gpt-5.4-mini (\$0.75 / \$4.50 per million prompt / completion tokens); cost is metered from billed usage, including calls whose output failed to parse. Generation, with 32 debates running concurrently, is the only paid stage; verification, evaluation, and backtesting are free.

¹Generation ran in two batches (180 and 276 ideas) under an identical configuration with a pause between them; we analyze the union as a single run.

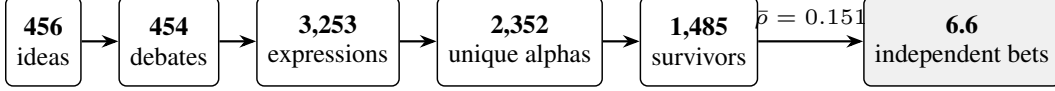


Figure 3: The run as an attrition chain. Two debates fail to complete; duplicate expressions collapse on normalized hash; the $|t| \geq 2$ gate removes weak candidates; and pairwise correlation compresses the 1,485 survivors into 6.6 effective independent bets.

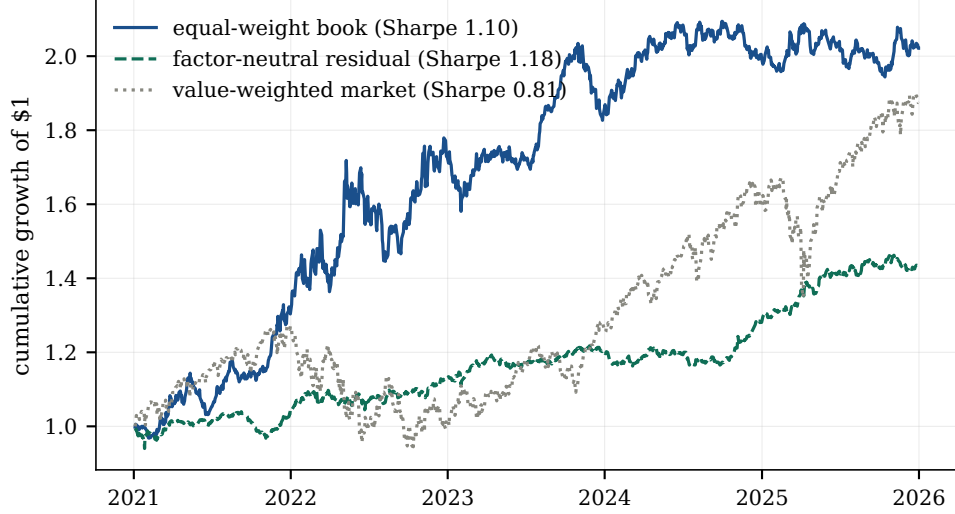


Figure 4: Test-split equity curves: the equal-weight book of all 1,485 survivors, its factor-neutral residual (11), and the value-weighted market over the same window. The residual grows more slowly but much more smoothly; its maximum drawdown is less than half the raw book’s.

5.1 Portfolio performance

Table 3 reports the test-split performance of the equal-weight book of all 1,485 survivors, its factor-neutral residual, and the single alpha with the strongest in-sample IC; Figure 4 plots the equity curves against the value-weighted market.

Book	n	IC	Sharpe	Ann. ret	MaxDD
best single alpha	1	0.0158	0.58	8.9%	−19.9%
equal-weight book	1,485	0.0165	1.10	15.2%	−15.9%
factor-neutral residual	1,485	0.0084	1.18	7.5%	−6.9%

Table 3: Out-of-sample performance on the test split (2021–2025), gross of transaction costs. The headline row is the equal-weight book; the residual row is the same book after the neutralization (11).

Two features of Table 3 matter most. First, the aggregate beats any single alpha: the strongest single alpha by in-sample IC earns a test Sharpe of 0.58, the book of all survivors 1.10, which is (10) operating as the formulation predicts. Second, the performance is not explained by static factor exposure. A book assembled from fundamental characteristics could earn its Sharpe by loading on well-known premia such as value or profitability, so each day we regress the composite cross-section on the per-day ranks of the full input characteristic basis (all ~ 46 fundamental panels, plus 60-day volatility and 21-day past return, missing ranks median-filled) and trade the residual:

$$\tilde{g}_t = g_t - X_t \hat{\beta}_t, \quad \hat{\beta}_t = \arg \min_{\beta} \|g_t - X_t \beta\|_2^2. \quad (11)$$

We use the full input basis because a small factor model is too coarse a control: a standard five-factor basis (size, value, quality, low-volatility, reversal levels) does not span signals built from a dozen correlated ratios. Neutralization removes roughly half the IC (0.0165 to 0.0084) and annual return

(15.2% to 7.5%), but the remaining half is far more risk-efficient, so the residual Sharpe, 1.18, is if anything higher than the raw book's, at less than half the drawdown (Figure 4).

5.2 What the generator wrote

Aggregate statistics say little about whether the debate produces economically sensible formulas, so we examine the outputs directly. Table 4 traces one debate on the theme *earnings quality and accruals* through its stages (3 idea drafts, 6 cross-reviews, 3 synthesized hypotheses, 12 formula drafts, 24 formula reviews, 7 selected formulas); Appendix A reproduces the raw artifact the trace is drawn from.

Stage 1 hypothesis	"Firms with strong earnings quality tend to convert accounting profits into durable operating performance, while high accrual intensity often reflects lower earnings persistence and greater subsequent reversals. The market can underreact to the distinction between cash-supported profitability and earnings inflated by non-cash accounting items."
Stage 2 draft	<code>add(sub(cs_rank(neg(cs_accruals)), cs_rank(cs_leverage)), cs_rank(cs_roa))</code>
Cross-review	" <code>cs_leverage</code> is not in the allowed terminal set for this run, so the formula is not strictly implementable as written." (accept with revision)
Revision	<code>add(add(cs_rank(neg(cs_accruals)), cs_rank(cs_roa)), cs_rank(neg(cs_debt_at)))</code>

Table 4: One debate, abridged: the synthesized hypothesis, a drafted formula, the cross-review that catches an unavailable terminal, and the revision that repairs it with an available one (`cs_debt_at`). Invalid fields are precisely the failure mode that one-shot generation does not catch before evaluation.

The run's modal survivor is the two-term expression

$$\text{sub}(\text{cs_rank}(\text{cs_roa}), \text{cs_rank}(\text{cs_accruals})),$$

long high-profitability, low-accrual names and short the reverse: the accruals anomaly of Sloan [15] combined with a profitability tilt. In-sample it has $t = 8.9$, IC 0.012, and turnover of 0.6% per day (fundamentals change slowly, so the signal is nearly static); out of sample it earns IC 0.0157 and Sharpe 1.16 on its own. The debate arrived at it honestly: the hypothesis in Table 4 is a fair statement of the earnings-persistence mechanism, and the reviewers graded the formula highly while noting it is "fairly conventional."

The deep end of the book. Not everything the generator writes is a linear blend of ranks. A short-term-reversal debate produced the three-way interaction

$$\text{mul}(\text{cs_rank}(\text{neg}(\text{ts_returns_5}(\text{close}))), \\ \text{mul}(\text{cs_rank}(\text{add}(\text{cs_roa}, \text{cs_roe})), \text{cs_rank}(\text{neg}(\text{cs_accruals}))))),$$

which buys recent five-day losers in proportion to their profitability and earnings quality: a conditional rule (buy quality on dips) rather than a static tilt. It carries its edge out of sample, with test IC 0.0177 and Sharpe 1.26 on its own, better than the modal quality alpha above. The contrast case is instructive. The highest in-sample IC in the entire pool (0.0185, $t = 8.7$) belongs to a depth-six interaction from a calendar-seasonality debate,

$$\text{mul}(\text{add}(\text{cs_rank}(\text{neg}(\text{returns})), \text{cs_rank}(\text{neg}(\text{abs_val}(\text{ts_returns_5}(\text{close}))))), \\ \text{add}(\text{cs_rank}(\text{dollar_volume}), \text{cs_rank}(\text{ts_rank_5}(\text{volume}))))),$$

a reversal signal conditioned on a quiet five-day path and concentrated in liquid, actively traded names. Built entirely from fast price and volume terminals, it turns over 25% of the book per day and decays to Sharpe 0.38 out of sample. In this pool, interactions whose legs are slow fundamentals kept their edge on the test split, while the fastest in-sample champions lost most of theirs.

The modal expression also illustrates the run's central weakness. Forty of the 454 debates (9%) selected this identical formula, reaching it from seven different themes, including *calendar / seasonality*;

it entered the book five times under algebraically equivalent spellings ($\text{sub}(a, b)$, $\text{add}(a, \text{neg}(b))$, and permutations) that the expression normalizer fails to unify, plus once sign-flipped. Convergence also operates within debates: in the firm-size debates, all three personas independently proposed the same market-capitalization rank. Terminal usage concentrates accordingly (Figure 5), with `cs_roa` alone appearing in nearly half of all surviving expressions. We ruled out response caching (all 454 hypothesis texts are distinct, no API seed is set, and decoding runs at the provider default temperature); the repetition is convergence. The theme string is the only input that varies across debates, so a strong model prior re-derives the same formula from many starting points.

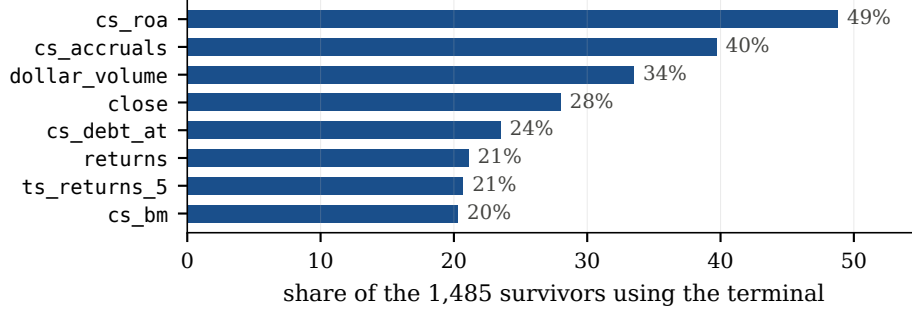


Figure 5: Terminal concentration among the 1,485 survivors. `cs_roa` appears in nearly half of all surviving expressions; the top four terminals between them touch most of the book.

5.3 Selection and effective breadth

Sweeping the selection gate over the full pool measures what in-sample significance buys out of sample:

$t_{\min}$	0	1	2	3	4
alphas retained	2,352	1,912	1,485	1,174	825
test Sharpe	1.02	1.11	1.10	1.17	1.16
composite IC	0.0134	0.0150	0.0165	0.0171	0.0172

Raising the bar improves both the composite IC and the test Sharpe up to $|t| \geq 3$, after which breadth loss flattens the curve. In-sample significance therefore carries genuine out-of-sample information for these alphas. The gate also purifies direction: the flip rate (Remark 3.3) falls from 32.0% across all verified alphas to 22.9% among survivors, well below the 50% of a direction-blind generator. Validity is 100%: every one of the 3,253 emitted expressions parses and verifies, the dividend of parse-gating inside the debate itself.

The book’s breadth, however, is far smaller than its alpha count. Equal-weighting N unit-variance signals with average pairwise correlation $\bar{\rho}$ gives book variance $(1/N)(1 + (N-1)\bar{\rho})$, so the effective number of independent bets is

$$N_{\text{eff}} = \frac{N}{1 + (N-1)\bar{\rho}} \xrightarrow{N \rightarrow \infty} \frac{1}{\bar{\rho}}. \quad (12)$$

Measured on the test split, the 1,485 survivors have $\bar{\rho} = 0.151$, giving $N_{\text{eff}} = 6.6$ against a ceiling of $1/\bar{\rho} = 6.6$: the book realizes 99.6% of the breadth attainable at its correlation level, and roughly fifty alphas already reach 90% of it (Figure 6). A checkpoint 40% of the way through the run (1,037 alphas, 667 survivors) was already at $N_{\text{eff}} = 6.1$, so the remaining \$60 of generation bought half an independent bet. This qualifies the breadth-first premise of Section 3: (10) rewards skill times the square root of breadth, but breadth is bounded above by $1/\bar{\rho}$, and the bound binds after the first hundred or so alphas. Past that point the only remaining lever is $\bar{\rho}$ itself, which is what Section 6 addresses.

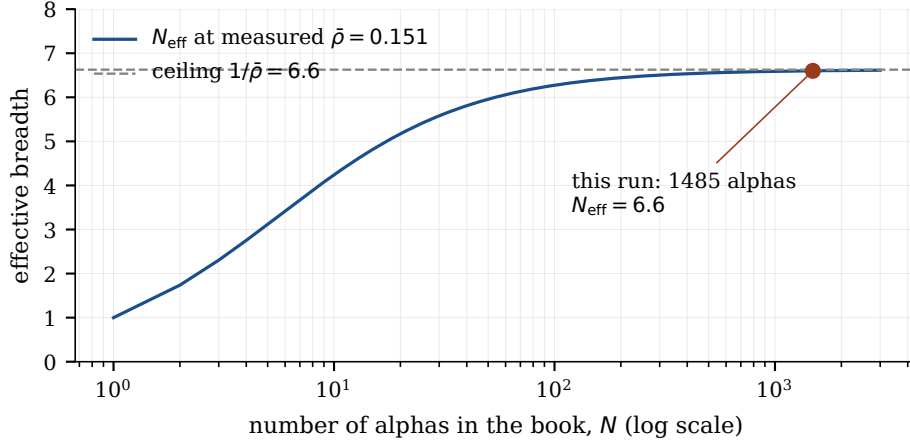


Figure 6: Effective breadth (12) as a function of book size at the run’s measured correlation. The curve saturates near fifty alphas; the full 1,485-survivor book sits at 99.6% of the ceiling. Adding alphas at this correlation level cannot add breadth.

6 Limitations and next steps

The results point to a clear bottleneck: performance is capped not by the number of alphas but by their average correlation, and the current generator’s $\bar{\rho} \approx 0.15$ caps effective breadth near seven independent bets. The natural response is to reduce $\bar{\rho}$ at generation time, before the expensive objective is ever queried, rather than to deduplicate after the fact.

Sample a terminal subset per idea. Every idea currently sees the full terminal menu, which lets a strong model prior steer many of them onto the same few fields. Instead, draw a random subset of y of the x available terminals for each idea. Two independently drawn y -subsets share y^2/x terminals in expectation, while the number of distinct subsets $\binom{x}{y}$ exceeds any feasible run size once $y \geq 6$ ($\binom{52}{6} \approx 2.0 \times 10^7$), so subset collisions are irrelevant and the overlap term alone should set y . Choosing $y \approx \sqrt{x}$ holds expected overlap near a single shared terminal; for the $x = 52$ menu used here that gives $y \approx 7$. Better still, sample across terminal *families* (price and volume, valuation, profitability, growth, leverage, liquidity) rather than uniformly, and occasionally withhold a theme’s obvious proxy. That makes the decorrelation mechanical rather than statistical: an idea never shown `market_cap` cannot rediscover `cs_rank(market_cap)`.

Remove the three fixed agents. The generator instantiates the same three personas (Momentum, MeanReversion, Fundamental) for every idea. The diagnostics in Section 5 show them converging on a single proposal rather than spanning three distinct ones. The fixed triple supplies cross-review pressure but little generative diversity, and it restricts every debate to three viewpoints chosen in advance. Two replacements seem worth testing. The first is to draw personas per idea from a much larger library. The second is to parameterize a persona along a few axes (horizon, direction, data family, transform style, nonlinearity) and sample a combination per idea. Either gives far more distinct viewpoints, while keeping the draft, review, and revise structure that the two-stage design depends on.

Further levers on $\bar{\rho}$. Themes are sampled with replacement from a fixed list, so the same theme recurs many times and yields near-clones; sampling without replacement, adding a sub-angle level, or generating the theme set dynamically would each help. The expression normalizer treats algebraically identical forms such as `sub(a,b)` and `add(a,neg(b))` as distinct, inflating the alpha count without adding information. Most ambitiously, generation could be made population aware: neutralize forward returns against the alphas already found and ask the next batch to explain the residual, which decorrelates by construction and terminates naturally when the residual stops shrinking.

Evaluation gaps. N_{eff} is computed on raw signals rather than on the factor-neutral residual, which is the stricter quantity, since alphas can be mutually uncorrelated while still loading on a single premium. All results are gross of transaction costs on one test window and one universe, and the 2021–2025 period was favorable to the characteristic families that dominate the book. Replication across windows, universes, and cost assumptions remains necessary before any of these Sharpe figures should be read as tradable.

7 Conclusion

We presented Alpha-GPT 3.0, a system in which LLM agents first argue out an economic idea in words and only then write it as a formula. Settling the argument first is what makes the output auditable. For any alpha in the book we can look up four things: the economic mechanism it was meant to encode, the agent that proposed it, the reviews it received, and the revision that produced its final form. Two further choices make the results interpretable. Everything after the generator is held fixed, so any measured difference comes from the formulas themselves. And every headline number is reported next to the same number computed after the portfolio’s exposure to the input characteristics has been removed. That comparison separates genuine signal from a portfolio that merely tracks well-known effects.

The experiments in Section 5 support the approach on performance. The book earns a test Sharpe of 1.10, and its factor-neutral residual holds 1.18 at less than half the drawdown. They also show where the current design stops working. Effective breadth saturates at $1/\bar{\rho} \approx 7$ independent bets. A checkpoint less than halfway through the run was already within half a bet of the final book, so the last \$60 of generation bought essentially nothing. Under (10) the design goal was breadth, but breadth is available only up to $1/\bar{\rho}$. The next step is therefore not to generate more alphas, but to generate less correlated ones. The terminal-subset sampling and the replacement of the fixed persona triple in Section 6 are both aimed at this. Other directions remain: transaction costs, replication across windows and universes, seeding GP with debate output at scale, the formula-versus-code ablation, and a feedback loop that lets agents see return-blind diagnostics of their earlier proposals.

References

- [1] D. Araci. FinBERT: Financial sentiment analysis with pre-trained language models. arXiv:1908.10063, 2019.
- [2] C.-M. Chan et al. ChatEval: Towards better LLM-based evaluators through multi-agent debate. ICLR, 2024.
- [3] W. Chen et al. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. ICLR, 2023.
- [4] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. arXiv:2305.14325, 2023.
- [5] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné. DEAP: Evolutionary algorithms made easy. JMLR 13:2171–2175, 2012.
- [6] S. Gu, B. Kelly, and D. Xiu. Empirical asset pricing via machine learning. Review of Financial Studies 33(5):2223–2273, 2020.
- [7] S. Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv:2308.00352, 2023.
- [8] Z. Kakushadze. 101 formulaic alphas. arXiv:1601.00991, 2016.
- [9] J. R. Koza. Genetic Programming: On the Programming of Computers by Means of Natural Selection. MIT Press, 1992.
- [10] G. Li et al. CAMEL: Communicative agents for “mind” exploration of large language model society. NeurIPS, 2023.
- [11] A. López-Lira and Y. Tang. Can ChatGPT forecast stock price movements? Return predictability and large language models. arXiv:2304.07619, 2023.

- [12] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS, 2023.
- [13] C. Qian et al. ChatDev: Communicative agents for software development. ACL, 2024.
- [14] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. arXiv:2303.11366, 2023.
- [15] R. G. Sloan. Do stock prices fully reflect information in accruals and cash flows about future earnings? *The Accounting Review* 71(3):289–315, 1996.
- [16] S. Wang, H. Yuan, L. Zhou, L. M. Ni, H.-Y. Shum, and J. Guo. Alpha-GPT: Human-AI interactive alpha mining for quantitative investment. arXiv:2308.00016, 2023.
- [17] X. Wang et al. Self-consistency improves chain of thought reasoning in language models. ICLR, 2023.
- [18] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. ICLR, 2022.
- [19] Q. Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.
- [20] S. Wu et al. BloombergGPT: A large language model for finance. arXiv:2303.17564, 2023.
- [21] H. Yang, X.-Y. Liu, and C. D. Wang. FinGPT: Open-source financial large language models. FinLLM Symposium at IJCAI, 2023.
- [22] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of Thoughts: Deliberate problem solving with large language models. NeurIPS, 2023.
- [23] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. ICLR, 2023.
- [24] H. Yuan, S. Wang, and J. Guo. Alpha-GPT 2.0: Human-in-the-loop AI for quantitative investment. arXiv:2402.09746, 2024.
- [25] F. Liu et al. Cognitive alpha mining via LLM-driven code-based evolution. arXiv:2511.18850, 2025.
- [26] Z. Kou et al. Automate strategy finding with LLM in quant investment. arXiv:2409.06289, 2024.
- [27] Z. Tang et al. AlphaAgent: LLM-driven alpha mining with regularized exploration to counteract alpha decay. arXiv:2502.16789, 2025.
- [28] Y. Shi, Y. Duan, and J. Li. Navigating the alpha jungle: an LLM-powered MCTS framework for formulaic factor mining. arXiv:2505.11122, 2025.
- [29] X. Li et al. R&D-Agent-Quant: a multi-agent framework for data-centric factors and model joint optimization. arXiv:2505.15155, 2025.
- [30] B. Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature* 625, 2024.
- [31] A. Novikov et al. AlphaEvolve: a coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025.
- [32] K.-H. Lee et al. Evolving deeper LLM thinking. arXiv:2501.09891, 2025.
- [33] S. Yu et al. Generating synergistic formulaic alpha collections via reinforcement learning. KDD, 2023.
- [34] T. Zhang et al. AutoAlpha: an efficient hierarchical evolutionary algorithm for mining alpha factors. arXiv:2002.08245, 2020.
- [35] C. Cui et al. AlphaEvolve: a learning framework to discover novel alphas in quantitative investment. SIGMOD, 2021.
- [36] H. Shi et al. AlphaForge: a framework to mine and dynamically combine formulaic alpha factors. AAAI, 2025.

A Example of a raw debate artifact

Every debate persists its full state as a structured JSON artifact (twelve files per idea: the brief, drafts, cross-reviews, revisions, synthesized hypotheses, formula drafts and reviews, and the selected seed pack), which is what makes the idea-to-formula chain of Table 4 auditable after the fact. The excerpt below is from the artifact behind Table 4 (theme *earnings quality and accruals*). All field values are reproduced from the actual run; long strings are elided with [...], wrapped lines are indented, and elided sibling entries are noted in brackets.

```
{
  "theme": "earnings quality and accruals",

  "hypotheses.json": [
    { "hypothesis_id": "hypothesis-earnings-quality-minus-accruals-with-mild-balance-sheet-support",
      "title": "Earnings Quality Minus Accruals with Mild Balance-Sheet Support",
      "source_agents": ["Fundamental", "Momentum"],
      "mechanism": "Firms with strong earnings quality tend to convert accounting profits into durable operating performance, while high accrual intensity often reflects lower earnings persistence and greater subsequent reversals. The market can underreact to the distinction between cash-supported profitability and earnings inflated by non-cash accounting items. [...]",
      "directionality": "Directional. Higher earnings quality and lower accruals should predict higher future returns [...]" },
    "[2 further hypotheses elided]"
  ],

  "formula_drafts.json": [
    { "formula_id": "formula-momentum-[...]-4",
      "agent_name": "Momentum",
      "formula_role": "directional_alpha",
      "expression": "add(sub(cs_rank(neg(cs_accruals)), cs_rank(cs_leverage)), cs_rank(cs_roa))",
      "plain_language_mapping": "Go long firms with low accruals and high ROA, and penalize firms with high leverage.",
      "expected_signal_direction": "Higher score indicates better earnings quality, stronger profitability, and lower balance-sheet stress." },
    "[11 further drafts elided]"
  ],

  "formula_reviews.json": [
    { "reviewer_agent_name": "MeanReversion",
      "target_formula_id": "formula-momentum-[...]-4",
      "faithfulness": 3, "implementability": 3, "robustness": 4,
      "novelty": 2, "simplicity": 5,
      "decision": "accept_with_revision",
      "comments": [
        "The core idea is faithful: reward low accruals and high ROA, penalize leverage. That is a sensible quality-minus-distress construction with contrarian/mean-reversion flavor.",
        "cs_leverage is not in the allowed terminal set for this run, so the formula is not strictly implementable as written." ] },
    "[23 further reviews elided]"
  ],

  "seed_formula_pack.json": {
    "pack_id": "seed-pack-selected",
    "selected_formulas": [
      "add(add(cs_rank(neg(cs_accruals)), cs_rank(cs_roa)), cs_rank(cs_roe))",
      "[6 further formulas elided]" ],
    "selection_rationale": [
      "Selected a clean, parser-safe main alpha for the compact profitability-confirmation hypothesis that stays faithful to the stated mechanism: low accruals plus cross-sectional profitability confirmation. It uses only supported terminals and matches the diagnostic profile of a stable slow-moving quality signal.",
      "[...]" ] }
  }
```

The structure enforces the discipline the method relies on: reviews are scored on fixed integer scales and attached to a specific target formula, revisions must record which feedback was accepted, and the moderator's selection rationale is recorded next to what it selected. The reviewer catching the unavailable `cs_leverage` terminal in this artifact is the same event abridged in Table 4.